

NAME:LOGESHWARI S

COURSECODE:CSA5112

REG NO:192565040

SLOT:B

<u>EXPERIMENT 1</u>

IMPLEMENTATION OF DES

AIM

To implement the Data Encryption Standard (DES) algorithm using Python.

ALGORITHM

Step-1: Start the program.

Step-2: Import the DES cipher and padding modules.

Step-3: Read the plaintext message from the user.

Step-4: Define an 8-byte secret key for DES encryption.

Step-5: Create the DES cipher in ECB mode using the secret key.

Step-6: Encrypt the plaintext after applying padding.

Step-7: Decrypt the ciphertext using the same secret key.

Step-8: Remove the padding from the decrypted text.

Step-9: Display the encrypted text and decrypted text.

Step-10: Stop the program.

PROGRAM (Python)

```
SUCCESS.py - C:/Users/cheth/SUCCESS.py (3.13.14)
File Edit Format Run Options Window Help
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

key = b'12345678'

plain = input("Enter Plain Text: ")

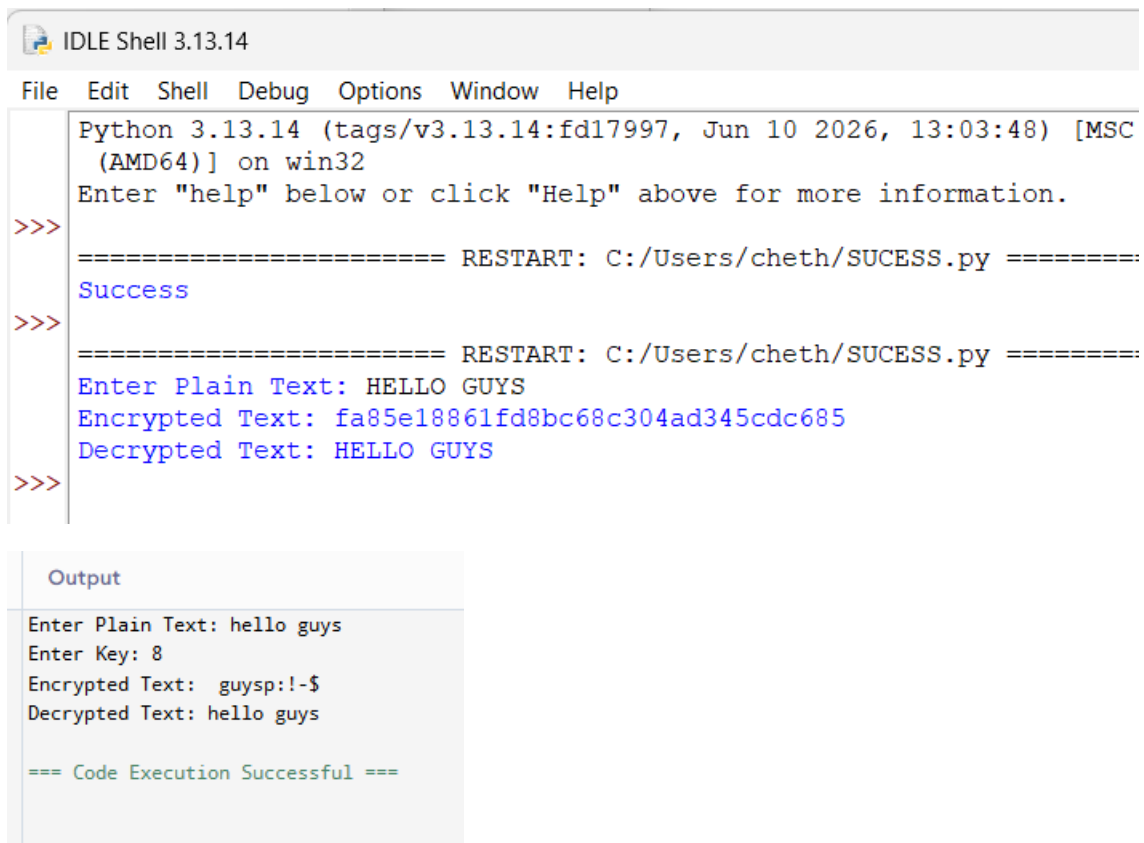
cipher = DES.new(key, DES.MODE_ECB)

encrypted = cipher.encrypt(pad(plain.encode(), 8))
print("Encrypted Text:", encrypted.hex())

decrypted = unpad(cipher.decrypt(encrypted), 8)
print("Decrypted Text:", decrypted.decode())
```

```
1- def feistel_encrypt(text, key, rounds):
2-     if len(text) % 2 != 0:
3-         text += " "
4-     mid = len(text) // 2
5-     left = text[:mid]
6-     right = text[mid:]
7-     for r in range(rounds):
8-         new_left = right
9-         new_right = ""
10-        for i in range(len(left)):
11-            f = chr((ord(right[i]) ^ key[r % len(key)]) % 256)
12-            new_right += chr(ord(left[i]) ^ ord(f))
13-        left = new_left
14-        right = new_right
15-    return left + right
16- def feistel_decrypt(cipher, key, rounds):
17-     mid = len(cipher) // 2
18-     left = cipher[:mid]
19-     right = cipher[mid:]
20-     for r in range(rounds - 1, -1, -1):
21-         old_right = left
22-         old_left = ""
23-         for i in range(len(right)):
24-             f = chr((ord(old_right[i]) ^ key[r % len(key)]) % 256)
25-             old_left += chr(ord(right[i]) ^ ord(f))
26-         left = old_left
27-         right = old_right
28-
29-     return (left + right).rstrip()
30 text = input("Enter Plain Text: ")
31 key = input("Enter Key: ")
32
33 key = [ord(c) for c in key]
34
35 cipher = feistel_encrypt(text, key, 4)
36 print("Encrypted Text:", cipher)
37
38 plain = feistel_decrypt(cipher, key, 4)
39 print("Decrypted Text:", plain)
```

OUTPUT



```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/cheth/SUCCESS.py =====
Success
>>>
===== RESTART: C:/Users/cheth/SUCCESS.py =====
Enter Plain Text: HELLO GUYS
Encrypted Text: fa85e18861fd8bc68c304ad345cdc685
Decrypted Text: HELLO GUYS
>>>

Output
Enter Plain Text: hello guys
Enter Key: 8
Encrypted Text: guysp!-$
Decrypted Text: hello guys

=== Code Execution Successful ===
```

RESULT

Thus, the Data Encryption Standard (DES) algorithm was implemented successfully, and the plaintext was encrypted and decrypted correctly using the secret key.

EXPERIMENT 2
IMPLEMENTATION OF 2DES (DOUBLE DATA ENCRYPTION STANDARD)

AIM

To implement a simplified Double Data Encryption Standard (2DES) algorithm using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext from the user.

Step 3: Read two encryption keys (Key 1 and Key 2).

Step 4: Encrypt the plaintext using Key 1.

Step 5: Encrypt the obtained ciphertext again using Key 2.

Step 6: Display the encrypted text.

Step 7: Decrypt the encrypted text using Key 2.

Step 8: Decrypt the obtained text using Key 1.

Step 9: Display the original plaintext.

Step 10: Stop the program.

PROGRAM (Python)

```
1- def encrypt(text, key):
2     result = ""
3-     for ch in text:
4         result += chr(ord(ch) ^ key)
5     return result
6
7- def decrypt(text, key):
8     result = ""
9-     for ch in text:
10        result += chr(ord(ch) ^ key)
11    return result
12
13 plain = input("Enter Plain Text: ")
14 key1 = int(input("Enter Key 1: "))
15 key2 = int(input("Enter Key 2: "))
16
17 cipher1 = encrypt(plain, key1)
18 cipher2 = encrypt(cipher1, key2)
19
20 print("Encrypted Text:", cipher2)
21
22 temp = decrypt(cipher2, key2)
23 original = decrypt(temp, key1)
24
25 print("Decrypted Text:", original)
```

OUTPUT

```
Output
Enter Plain Text: HELLO EVERYONE
Enter Key 1: 5
Enter Key 2: 9
Encrypted Text: DI@@C,IZI^UCBI
Decrypted Text: HELLO EVERYONE

=== Code Execution Successful ===
```

RESULT

Thus, the simplified Double Data Encryption Standard (2DES) algorithm was implemented successfully, and the plaintext was encrypted and decrypted correctly using two different keys.

EXPERIMENT 3
IMPLEMENTATION OF 3DES (TRIPLE DATA ENCRYPTION STANDARD)

AIM

To implement a simplified Triple Data Encryption Standard (3DES) algorithm using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext from the user.

Step 3: Read three encryption keys (Key 1, Key 2, and Key 3).

Step 4: Encrypt the plaintext using Key 1.

Step 5: Decrypt the encrypted text using Key 2.

Step 6: Encrypt the obtained text again using Key 3.

Step 7: Display the encrypted text.

Step 8: Decrypt the ciphertext using Key 3.

Step 9: Encrypt the obtained text using Key 2.

Step 10: Decrypt the obtained text using Key 1 to obtain the original plaintext.

Step 11: Display the decrypted text.

Step 12: Stop the program.

PROGRAM (Python)

```
main.py
1- def encrypt(text, key):
2     result = ""
3-     for ch in text:
4         result += chr(ord(ch) ^ key)
5     return result
6
7- def decrypt(text, key):
8     result = ""
9-     for ch in text:
0         result += chr(ord(ch) ^ key)
1     return result
2
3 plain = input("Enter Plain Text: ")
4 key1 = int(input("Enter Key 1: "))
5 key2 = int(input("Enter Key 2: "))
6 key3 = int(input("Enter Key 3: "))
7
8 # 3DES Encryption (E-D-E)
9 step1 = encrypt(plain, key1)
0 step2 = decrypt(step1, key2)
1 cipher = encrypt(step2, key3)
2
3 print("Encrypted Text:", cipher)
4
5 # 3DES Decryption
6 step3 = decrypt(cipher, key3)
7 step4 = encrypt(step3, key2)
8 original = decrypt(step4, key1)
9
0 print("Decrypted Text:", original)
```

OUTPUT

Output
Enter Plain Text: CHETHRIKA Enter Key 1: 7 Enter Key 2: 6 Enter Key 3: 4 Encrypted Text: FM@QMWLND Decrypted Text: CHETHRIKA === Code Execution Successful ===

RESULT

Thus, the simplified Triple Data Encryption Standard (3DES) algorithm was implemented successfully, and the plaintext was encrypted and decrypted correctly using three different keys.

EXPERIMENT 4(A)
IMPLEMENTATION OF DES – ECB (Electronic Code Book) MODE

AIM

To implement the Electronic Code Book (ECB) mode of DES using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext and encryption key from the user.

Step 3: Divide the plaintext into individual blocks.

Step 4: Encrypt each block independently using the key.

Step 5: Display the encrypted text.

Step 6: Decrypt the encrypted text using the same key.

Step 7: Display the original plaintext.

Step 8: Stop the program.

PROGRAM(PYTHON)

```
1- def encrypt(text, key):
2     result = ""
3-     for ch in text:
4         result += chr(ord(ch) ^ key)
5     return result
6
7- def decrypt(text, key):
8     result = ""
9-     for ch in text:
10        result += chr(ord(ch) ^ key)
11    return result
12
13 plain = input("Enter Plain Text: ")
14 key = int(input("Enter Key: "))
15
16 cipher = encrypt(plain, key)
17 print("Encrypted Text:", cipher)
18
19 text = decrypt(cipher, key)
20 print("Decrypted Text:", text)
```

OUTPUT

Output
Enter Plain Text: HI GUYS Enter Key: 8 Encrypted Text: @A(0]Q[Decrypted Text: HI GUYS === Code Execution Successful ===

RESULT

Thus, the simplified DES Electronic Code Book (ECB) mode was implemented successfully, and the plaintext was encrypted and decrypted correctly.

EXPERIMENT 4(B)
IMPLEMENTATION OF DES – CBC (Cipher Block Chaining) MODE

AIM

To implement the Cipher Block Chaining (CBC) mode of DES using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext, key, and Initialization Vector (IV).

Step 3: Initialize the previous block with the IV.

Step 4: XOR the plaintext character with the previous block.

Step 5: Encrypt the result using the key.

Step 6: Store the encrypted character as the current ciphertext.

Step 7: Use the current ciphertext as the previous block for the next character.

Step 8: Repeat Steps 4–7 until all characters are encrypted.

Step 9: Display the encrypted text.

Step 10: Decrypt the ciphertext using the same key and IV.

Step 11: Display the decrypted text.

Step 12: Stop the program.

PROGRAM(PYTHON)

```
1- def encrypt(text, key, iv):
2     cipher = ""
3     prev = iv
4
5-     for ch in text:
6         c = (ord(ch) ^ prev) ^ key
7         cipher += chr(c)
8         prev = c
9
10    return cipher
11
12
13- def decrypt(cipher, key, iv):
14     plain = ""
15     prev = iv
16
17-     for ch in cipher:
18         p = (ord(ch) ^ key) ^ prev
19         plain += chr(p)
20         prev = ord(ch)
21
22    return plain
23
24
25 plain = input("Enter Plain Text: ")
26 key = int(input("Enter Key: "))
27 iv = int(input("Enter IV: "))
28
29 cipher = encrypt(plain, key, iv)
30 print("Encrypted Text:", cipher)
31
32 text = decrypt(cipher, key, iv)
33 print("Decrypted Text:", text)
```

OUTPUT

```
Enter Plain Text: computer
Enter Key: 5
Enter IV: 10
Encrypted Text: 1.n.z
Decrypted Text: computer

=== Code Execution Successful ===
```

RESULT

Thus, the simplified DES Cipher Block Chaining (CBC) mode was implemented successfully, and the plaintext was encrypted and decrypted correctly.

EXPERIMENT 4(c)
IMPLEMENTATION OF DES – CFB (Cipher Feedback) MODE

AIM

To implement the Cipher Feedback (CFB) mode of DES using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext, key, and Initialization Vector (IV).

Step 3: Initialize the feedback value with the IV.

Step 4: Encrypt the feedback value using the key.

Step 5: XOR the encrypted feedback with the plaintext to generate the ciphertext.

Step 6: Use the generated ciphertext as the new feedback value.

Step 7: Repeat the process until all characters are encrypted.

Step 8: Display the encrypted text.

Step 9: Decrypt the ciphertext using the same key and IV.

Step 10: Display the decrypted text.

Step 11: Stop the program.

PROGRAM (Python)

```
1- def encrypt(text, key, iv):
2     cipher = ""
3     feedback = iv
4-     for ch in text:
5         temp = feedback ^ key
6         c = ord(ch) ^ temp
7         cipher += chr(c)
8         feedback = c
9
10    return cipher
1- def decrypt(cipher, key, iv):
2     plain = ""
3     feedback = iv
4
5-     for ch in cipher:
6         temp = feedback ^ key
7         p = ord(ch) ^ temp
8         plain += chr(p)
9         feedback = ord(ch)
10    return plain
1 plain = input("Enter Plain Text: ")
2 key = int(input("Enter Key: "))
3 iv = int(input("Enter IV: "))
4
5 cipher = encrypt(plain, key, iv)
6
7 print("Encrypted Text:", cipher)
8
9 text = decrypt(cipher, key, iv)
10
11 print("Decrypted Text:", text)
```

OUTPUT

In	Output
	<pre>Enter Plain Text: happy Enter Key: 5 Enter IV: 7 Encrypted Text: j·{·r Decrypted Text: happy === Code Execution Successful ===</pre>

RESULT

Thus, the simplified DES Cipher Feedback (CFB) mode was implemented successfully, and the plaintext was encrypted and decrypted correctly.

EXPERIMENT 4(D)
IMPLEMENTATION OF DES – OFB (Output Feedback) MODE

AIM

To implement the Output Feedback (OFB) mode of DES using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext, key, and Initialization Vector (IV).

Step 3: Initialize the feedback value with the IV.

Step 4: Encrypt the feedback value using the key.

Step 5: XOR the encrypted feedback with the plaintext to generate the ciphertext.

Step 6: Use the encrypted feedback (not the ciphertext) as the feedback for the next round.

Step 7: Repeat the process until all characters are encrypted.

Step 8: Display the encrypted text.

Step 9: Decrypt the ciphertext using the same key and IV.

Step 10: Display the decrypted text.

Step 11: Stop the program.

PROGRAM (Python)

```
main.py
1- def encrypt(text, key, iv):
2     cipher = ""
3     feedback = iv
4-     for ch in text:
5         feedback = feedback ^ key
6         c = ord(ch) ^ feedback
7         cipher += chr(c)
8     return cipher
9- def decrypt(cipher, key, iv):
10    plain = ""
11    feedback = iv
12-    for ch in cipher:
13        feedback = feedback ^ key
14        p = ord(ch) ^ feedback
15        plain += chr(p)
16    return plain
17 plain = input("Enter Plain Text: ")
18 key = int(input("Enter Key: "))
19 iv = int(input("Enter IV: "))
20
21 cipher = encrypt(plain, key, iv)
22 print("Encrypted Text:", cipher)
23
24 text = decrypt(cipher, key, iv)
25 print("Decrypted Text:", text)
```

OUTPUT

run	Output
	Enter Plain Text: java program Enter Key: 6 Enter IV: 7 Encrypted Text: kfwf!wshfu`j Decrypted Text: java program === Code Execution Successful ===

RESULT

Thus, the simplified DES Output Feedback (OFB) mode was implemented successfully, and the plaintext was encrypted and decrypted correctly.

EXPERIMENT 4(E)
IMPLEMENTATION OF DES – OFB (Output Feedback) MODE

AIM

To implement the Counter (CTR) mode of DES using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext, key, and counter value from the user.

Step 3: Initialize the counter.

Step 4: Encrypt the counter using the key.

Step 5: XOR the encrypted counter with the plaintext to generate the ciphertext.

Step 6: Increment the counter for the next character.

Step 7: Repeat Steps 4–6 until all characters are encrypted.

Step 8: Display the encrypted text.

Step 9: Reset the counter and decrypt the ciphertext using the same key.

Step 10: Display the decrypted text.

Step 11: Stop the program.

PROGRAM (Python)

```
main.py
1- def encrypt(text, key, counter):
2     cipher = ""
3
4-     for ch in text:
5         c = ord(ch) ^ (counter ^ key)
6         cipher += chr(c)
7         counter += 1
8
9     return cipher
10
11
12- def decrypt(cipher, key, counter):
13     plain = ""
14
15-     for ch in cipher:
16         p = ord(ch) ^ (counter ^ key)
17         plain += chr(p)
18         counter += 1
19
20     return plain
21
22
23 plain = input("Enter Plain Text: ")
24 key = int(input("Enter Key: "))
25 counter = int(input("Enter Counter Value: "))
26
27 cipher = encrypt(plain, key, counter)
28 print("Encrypted Text:", cipher)
29
30 text = decrypt(cipher, key, counter)
31 print("Decrypted Text:", text)
```

OUTPUT

Output
Enter Plain Text: happy days
Enter Key: 6
Enter Counter Value: 8
Encrypted Text: fn }s+lhod
Decrypted Text: happy days
=== Code Execution Successful ===

RESULT

Thus, the simplified DES Counter (CTR) mode was implemented successfully, and the plaintext was encrypted and decrypted correctly.

EXPERIMENT 5
IMPLEMENTATION OF RSA ALGORITHM

AIM

To implement the RSA algorithm for encryption and decryption using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read two prime numbers (p and q) from the user.

Step 3: Compute $n = p \times q$.

Step 4: Compute Euler's Totient Function $\phi(n) = (p - 1) \times (q - 1)$.

Step 5: Choose the public key e such that it is relatively prime to $\phi(n)$.

Step 6: Compute the private key d.

Step 7: Read the plaintext message.

Step 8: Encrypt the plaintext using the public key.

Step 9: Display the encrypted message.

Step 10: Decrypt the encrypted message using the private key.

Step 11: Display the decrypted message.

Step 12: Stop the program.

PROGRAM (Python)

```
1- def gcd(a, b):
2-     while b != 0:
3-         a, b = b, a % b
4-     return a
5
6 p = int(input("Enter Prime Number p: "))
7 q = int(input("Enter Prime Number q: "))
8
9 n = p * q
10 phi = (p - 1) * (q - 1)
11
12 e = 2
13 while e < phi:
14     if gcd(e, phi) == 1:
15         break
16     e += 1
17
18 d = 1
19 while (d * e) % phi != 1:
20     d += 1
21
22 msg = int(input("Enter Message (number): "))
23
24 cipher = (msg ** e) % n
25 print("Encrypted Message:", cipher)
26
27 plain = (cipher ** d) % n
28 print("Decrypted Message:", plain)
```

OUTPUT

```
Enter Prime Number p: 11
Enter Prime Number q: 13
Enter Message (number): 9
Encrypted Message: 48
Decrypted Message: 9

=== Code Execution Successful ===
```

RESULT

Thus, the RSA algorithm was implemented successfully, and the plaintext was encrypted and decrypted correctly.

EXPERIMENT 6

IMPLEMENTATION OF DIFFIE–HELLMAN KEY EXCHANGE ALGORITHM

AIM

To implement the Diffie–Hellman Key Exchange Algorithm using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the prime number (P) and primitive root (G).

Step 3: Read the private keys of User A and User B.

Step 4: Compute the public key of User A.

Step 5: Compute the public key of User B.

Step 6: Exchange the public keys.

Step 7: Compute the secret key at User A.

Step 8: Compute the secret key at User B.

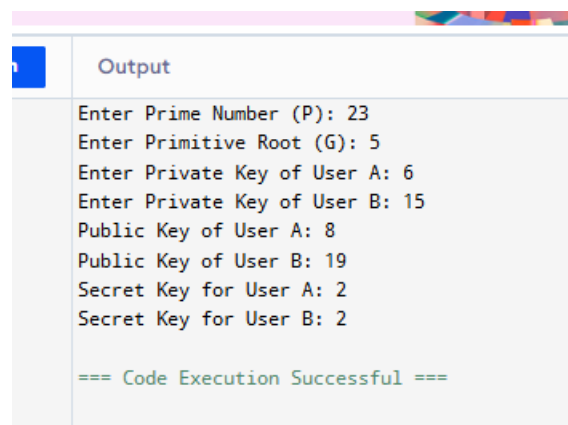
Step 9: Display both secret keys.

Step 10: Stop the program.

PROGRAM (Python)

```
main.py
1 P = int(input("Enter Prime Number (P): "))
2 G = int(input("Enter Primitive Root (G): "))
3
4 a = int(input("Enter Private Key of User A: "))
5 b = int(input("Enter Private Key of User B: "))
6
7 A = (G ** a) % P
8 B = (G ** b) % P
9
10 print("Public Key of User A:", A)
11 print("Public Key of User B:", B)
12
13 keyA = (B ** a) % P
14 keyB = (A ** b) % P
15
16 print("Secret Key for User A:", keyA)
17 print("Secret Key for User B:", keyB)
```

OUTPUT



```
Output
Enter Prime Number (P): 23
Enter Primitive Root (G): 5
Enter Private Key of User A: 6
Enter Private Key of User B: 15
Public Key of User A: 8
Public Key of User B: 19
Secret Key for User A: 2
Secret Key for User B: 2

=== Code Execution Successful ===
```

RESULT

Thus, the Diffie–Hellman Key Exchange Algorithm was implemented successfully, and both users generated the same shared secret key.

EXPERIMENT 7

IMPLEMENTATION OF ELLIPTIC CURVE CRYPTOGRAPHY (ECC)

AIM

To implement the Elliptic Curve Cryptography (ECC) algorithm using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the prime number (P), generator point (G), and private keys of User A and User B.

Step 3: Generate the public key of User A by multiplying the generator point with the private key.

Step 4: Generate the public key of User B by multiplying the generator point with the private key.

Step 5: Exchange the public keys between both users.

Step 6: Compute the shared secret key at User A using User B's public key.

Step 7: Compute the shared secret key at User B using User A's public key.

Step 8: Display the public keys and the shared secret keys.

Step 9: Stop the program.

PROGRAM (Python)

```
main.py
1 P = int(input("Enter Prime Number (P): "))
2 G = int(input("Enter Generator Point (G): "))
3
4 a = int(input("Enter Private Key of User A: "))
5 b = int(input("Enter Private Key of User B: "))
6
7 publicA = (a * G) % P
8 publicB = (b * G) % P
9
10 print("Public Key of User A:", publicA)
11 print("Public Key of User B:", publicB)
12
13 secretA = (a * publicB) % P
14 secretB = (b * publicA) % P
15
16 print("Shared Secret Key (User A):", secretA)
17 print("Shared Secret Key (User B):", secretB)
```

OUTPUT

Output
Enter Prime Number (P): 23 Enter Generator Point (G): 5 Enter Private Key of User A: 6 Enter Private Key of User B: 8 Public Key of User A: 7 Public Key of User B: 17 Shared Secret Key (User A): 10 Shared Secret Key (User B): 10 === Code Execution Successful ===

RESULT

Thus, the Elliptic Curve Cryptography (ECC) algorithm was implemented successfully, and both users generated the same shared secret key.